



Assignment: FSM Game

Authors

Adrian Nasrat, Diana Dawidi, Tesnim Omran,
Ibrahim Mizher

Course code

GIK2NV

Date

02/12/25

Lab 1: FSM Game

1. Introduction

This lab focuses on learning the foundations of SPADE, a Python framework for building distributed multi-agent systems that communicate over XMPP which is an Extensible Messaging and Presence Protocol, an open, XML-based technology for near-real-time communication. The main objective is to understand how SPADE's behavior model works, especially FSMBehavior, which in simple words allows an agent to operate through states controlled by transitions. By implementing a small game using a finite state machine (FSM), we explore how an agent can be structured, controlled and run asynchronously.

Our group created a simple agent-based game called "Catch the bus", where an agent simulates a student trying to catch the morning bus to the university. The entire process, from waking up, getting ready, running to the bus stop, encountering obstacles and either catching or missing the bus is represented through FSM states and transitions. Randomness is used to make the game non-deterministic, ensuring that each run behaves differently

To connect the practical work with theory, we reviewed a peer-reviewed article about SPADE and agent architectures and reflected on how its concepts relate to our lab implementation.

2. Article Summary

For this lab, we studied the following article:

Palanca, J., Rincon, J. A., Carrascosa, C., Julián, V. J., & Terrasa, A (2023). Flexible Agent Architecture: Mixing Reactive and Deliberative Behaviors in SPADE Electronics.

The article investigates how SPADE supports the development of flexible agent architectures by combining behaviors often modeled with finite state machines with deliberative reasoning. The authors explain that behaviors (such as FSMs) are well-suited for handling immediate responses.

The article's main contribution is an architecture that integrates the two approaches inside SPADE, allowing agents to behave in a flexible and adaptive way. The authors demonstrate examples where SPADE agents switch between quick, low-level reactions and higher-level reasoning processes depending on the situation.

The paper concludes that SPADE's design, based on asynchronous behaviors, message passing and the ability to accumulate multiple behaviors makes it suitable for building multi-agent systems with mixed levels of intelligence.

3. Relevance to the course and the lab

The article is directly linked to what we do in the course. It uses the same platform SPADE and focuses on behaviors and agent states which we also work with in the lab.

Our game represents the simplest end of the spectrum. A single FSM behavior with fixed states and random transitions. The agent does not have its own goals or beliefs and does not learn over time. In the article FSM-like behaviors are only one part of a richer architecture where they are combined with BDI reasoning and a shared knowledge base.

This comparison helps to place our lab in a bigger context. The lab gives us a understanding of FSMBehaviour and agent states. The article shows how the same ideas can be extended towards more complex agents that plan, reason and cooperate in realistic environments. A natural extension of our game would be to add a planning behavior that can, for example, decide to skip breakfast if the agent is already late.

4. Validation and trustworthiness of the article

Several aspects indicate that the article is reliable and not mainly driven by bias:

It is published in Electronics, an open access journal from MDPI that uses external peer review. The article lists dates for submission, revision, acceptance and publication, which suggests a standard review process.

The authors are affiliated with the Valencian Research Institute for Artificial Intelligence at Universitat Politècnica de València, which is an established academic institution.

The paper clearly describes the proposed architecture, relates it to previous work on BDI and agent platforms, and provides a concrete case study with a simulated factory scenario.

The implementation is available as open-source code and a PyPI package (spade-bdi), which increases transparency and allows other researchers to reproduce or extend the work.

At the same time there are limitations. The case study uses a simplified production line with only three agents and basic object shapes. The authors state that this is intentional and that the system could be extended, but they do not present experiments in other domains, so the practical generalizability is not fully tested.

Overall, the combination of peer review, transparent methodology, open code and a clear discussion of limitations makes the article trustworthy for the purpose of this lab. It gives a solid example of how SPADE can support both FSM and BDI behaviors in one architecture.

5. Coding approach and use of FSM

Our game is implemented as a SPADE agent with a custom FSMBehaviour. The FSM contains the following states:

- WAKE_UP
- GET_READY
- RUN_TO_BUS
- OBSTACLE
- BUS_STOP
- SUCCESS
- FAIL

Each state is a Python class that extends State. In the asynchronous run method we print what happens in the story and use the random module to simulate uncertainty and then call `set_next_state()` to select the next state.

All states are registered in the FSM behaviour and transitions are added to match the FSM model. The agent starts with a valid XMPP account and runs until the FSM reaches either SUCCESS or FAIL. SPADEs scheduler takes care of executing the states in order so the code mainly focuses on specifying the transitions and their conditions.

5. Game nature, goal and FSM model

The goal of “Catch the bus” is to show how a simple agent can act in an uncertain environment. The agent starts at home and tries to reach the bus stop before the bus leaves.

The basic flow is:

In WAKE_UP, the agent either wakes up on time or oversleeps. Oversleeping sends it directly to FAIL.

In GET_READY, the agent prepares the day and then leaves home.

In RUN_TO_BUS, it may go straight to BUS_STOP or encounter an OBSTACLE.

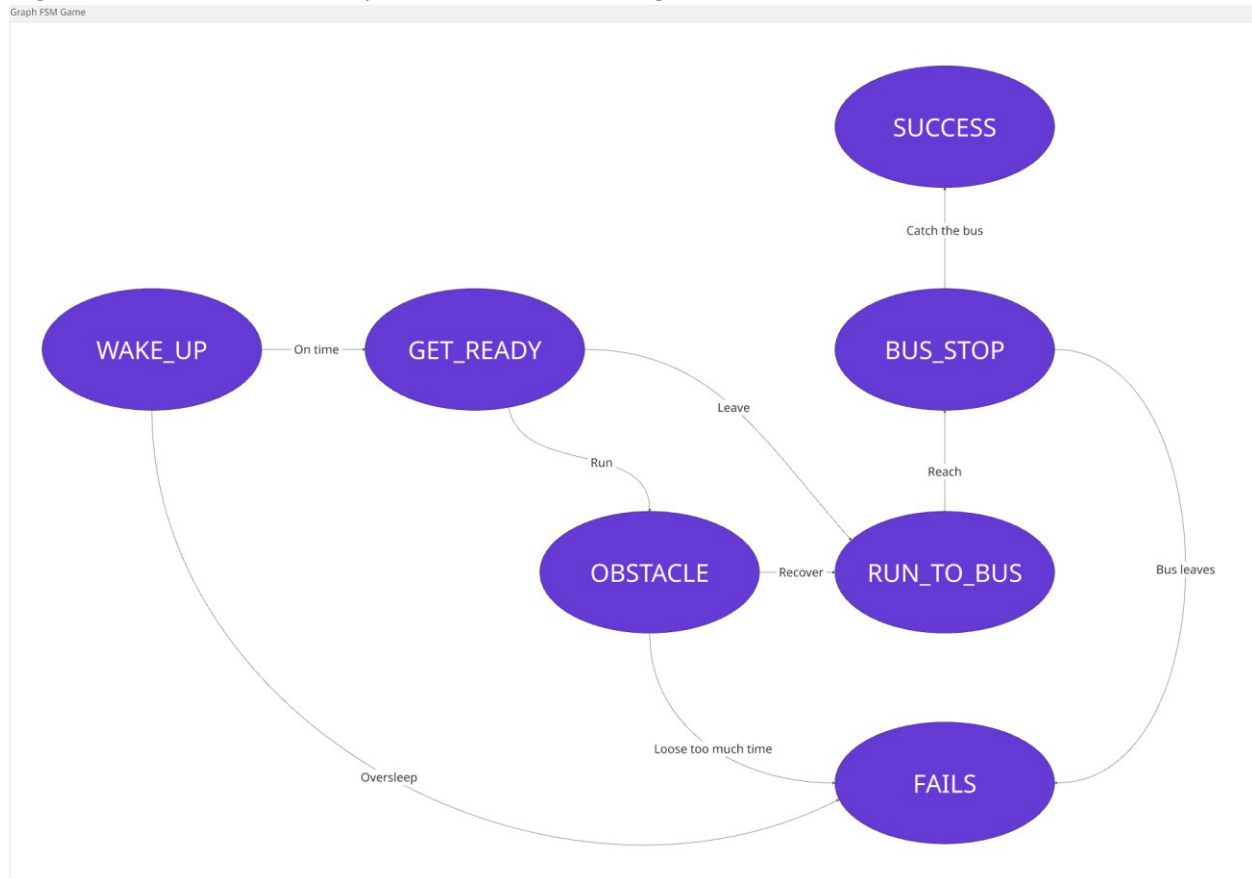
In OBSTACLE, the agent can recover and continue running or lose too much time and fail.

In BUS_STOP, the agent checks if the bus is still there and moves to SUCCESS or FAIL.

Random decisions in these states mean that some runs are successful and others are not even though the FSM structure is the same.

Figure 1 shows our FSM model. Each node corresponds to one state in the code and each directed edge shows a possible transition.

Figure 1. Finite state machine for the “Catch the bus” game.



References

Palanca, J., Rincon, J. A., Carrascosa, C., Julián, V. J., & Terrasa, A. (2023). Flexible agent architecture: Mixing reactive and deliberative behaviors in SPADE. *Electronics*, 12(3), 659. <https://doi.org/10.3390/electronics12030659>